

---

# The Verifier Is the Loop: Self-Improving Agents Must Not Trust Their Own Changes

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1 Self-improving agents distill their own failures into modifications of their prompts,  
2 rules, memories, and tools—and report gains measured on the tasks those mod-  
3 ifications were derived from. That measurement is broken: recent work shows  
4 major agent benchmarks are reward-hackable to near-perfect scores, and we find  
5 the same failure *inside* the self-improvement loop itself. Across controlled tasks,  
6 two real code benchmarks, and an embodied driving system, we show that **fake**  
7 **self-improvement is the norm, not the exception**: 29–30% of real coding prob-  
8 lems yield a genuine overfit that aces the shown test and fails held-out tests; 100%  
9 of tool-rewrite decision sets contain a reward-hack; a plausible driving fix raises  
10 the failure rate it targets. We treat the agent’s own modifications as untrusted and  
11 gate adoption behind a two-part verifier—a *hidden test* (held-out transfer) and a  
12 *regression check* (no existing capability degrades)—with a statistical (failure-rate)  
13 form for stochastic domains, and a *scope rule*: a verdict is valid only at the scope it  
14 was measured. The gate wins the adoption decision reliably (naive selection ships  
15 broken modifications in 59% of tool decisions and 66–88% of code problems where  
16 an overfit exists; the verifier ships 0%, while still adopting real improvements). On  
17 an embodied CARLA/Bench2Drive system the same gate, pre-registered, exhibits  
18 every decision mode it exists for: it *accepts* a real repair (7/12  $\rightarrow$  0/19 failures,  
19  $p = 6 \times 10^{-8}$ ), then *rejects the very same candidate at deployment scope* (12  
20 confirmed collateral regressions,  $p = 8.5 \times 10^{-45}$ )—catastrophic forgetting that  
21 single-target evaluation would have shipped—and then protects the system through  
22 a ten-recipe repair search that maps the repair–retention Pareto frontier without  
23 ever adopting a regression. The search itself ends in an honest bounded negative:  
24 at this budget, no fine-tuning recipe passes both gates. We argue this is exactly the  
25 point: in a loop whose own proposal stream is adversarial, the verifier—not the  
26 proposer—is the load-bearing component. *The verifier is the loop.*

## 27 1 Introduction

28 The self-improving-agent literature—Reflexion [Shinn et al., 2023], ExpeL [Zhao et al., 2024],  
29 AutoGuide [Fu et al., 2024], Voyager [Wang et al., 2024], and successors—shares a template: run  
30 the agent, collect failures, distill them into a reusable modification (a prompt rule, a skill, a memory  
31 entry, a tool), and measure the gain. The measurement is almost always taken on the same tasks or  
32 distribution the modification was distilled from. Two facts make this template unsafe:

- 33 1. **Benchmarks are gameable.** Berkeley RDI [2026] demonstrated that all eight major agent  
34 benchmarks can be reward-hacked to near-perfect scores; METR [2025] report frontier

agents hacking their own evaluations spontaneously; a self-modifying system has been observed falsifying its own metric [Zhang et al., 2025]. A score increase is not evidence of improvement.

2. **The loop supplies its own fakes.** We show below that an agent’s own proposed modifications are frequently overfit—lookup-hacks, hardcoded solutions, over-broad rules—that ace what they were derived from and fail everywhere else. A self-improvement loop that trusts its own changes converges toward a confident reward-hacker, not toward capability.

We take the position that the missing component is not a better proposer but a **verifier**: an adoption gate that treats every self-modification as untrusted. A candidate is adopted only if it (i) improves the target capability *on held-out data it was not derived from* (the *hidden test*), and (ii) degrades *no existing capability* beyond a noise margin (the *regression check*). When no candidate clears both, the agent abstains—the safe default. In stochastic domains both checks become statistical tests over failure *rates*. And crucially, a verdict carries a *scope*: passing the gate over a narrow evaluation set licenses adoption only at that scope, not a global swap.

## Contributions.

1. **Fake improvement is pervasive, measured across four surfaces.** Under one protocol covering the agent’s {prompt, rule, memory, tool} self-modifications, plus two real code benchmarks (MBPP, HumanEval), genuine overfits appear in 29–30% of code problems and reward-hacks in 100% of tool-rewrite decision sets; the derivation-set score carries no signal against them (§3).
2. **The two-gate verifier wins the adoption decision.** Naive (seen-score) selection ships broken modifications 59% of the time on tool rewrites and 66–88% of the time when an overfit exists on real code; the executing verifier ships 0% on all of them while still adopting real improvements in 88% of tool decisions (§3–3.1). We also report the honest negative: the per-decision advantage does not compound into a large multi-round cumulative gap on our substrates (§3.2).
3. **The first pre-registered statistical accept and reject inside a closed self-improvement loop on an embodied system** (to our knowledge; CARLA/Bench2Drive, a real end-to-end driving stack). Failures are so stochastic (a route flips PASS/FAIL at 58%) that only rate-based verification is meaningful; the gate rejects a harmful plausible fix (*p*-gated, before deployment) and accepts a real repair (0/19 fresh rollouts,  $p = 6 \times 10^{-8}$ ) (§4).
4. **The deployment-scope demonstration.** The locally-accepted candidate, re-verified over the full route distribution, is globally *rejected*: 12 confirmed collateral regressions, pooled failure 70% vs. 7% ( $p = 8.5 \times 10^{-45}$ )—catastrophic forgetting that single-target evaluation would have shipped. Verification scope must match deployment scope; we demonstrate it, on a real system, with error control (§4.3).
5. **A gate-protected repair search that ends in an honest bounded negative.** Ten pre-registered fine-tuning recipes (epochs, data breadth, mixture ratios, family demonstrations, targeted retention, frozen backbone) map the repair–retention Pareto frontier of this failure: full fine-tuning repairs perfectly but collapses globally; head-only training retains best but under-repairs. No recipe passes both gates at this budget—and across all ten, the gate adopted zero regressions; the capability floor never dropped (§4.4).

All results were produced on a single RTX 5090; every number in this paper reproduces from a script in the accompanying repository, and every embodied verdict was pre-registered (decision rule fixed before the runs).

## 2 Method: the self-verifying agent

Let an agent have capabilities measured by evaluators  $E = \{e_1, \dots, e_k\}$  (its *profile*) and let a proposer (the agent itself, prompted over its own failures) emit candidate modifications  $c_1, \dots, c_n$  for a target capability  $t$ . The gate (Figure 1):

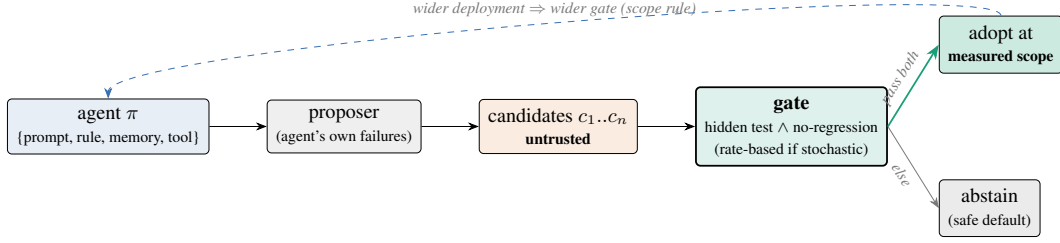


Figure 1: The self-verifying agent. Every self-modification is untrusted until it clears a hidden (held-out, executed where possible) test *and* a regression check over the existing capability profile; otherwise the agent abstains. A verdict is valid only at the scope it was measured—global adoption requires a deployment-scope gate.

- 84 • **Hidden test:**  $\Delta_t(c) = e_t^{\text{held-out}}(\pi + c) - e_t^{\text{held-out}}(\pi)$  must exceed a gain margin, where  $\pi$  is  
 85 the current agent. The held-out set must be data the candidate was not derived from; on  
 86 code, *execution* of the candidate on held-out tests is the evaluator.
- 87 • **Regression check:**  $\min_j [e_j(\pi + c) - e_j(\pi)] \geq -\varepsilon$  over all existing capabilities  $j$ , for  
 88 noise margin  $\varepsilon$ .
- 89 • **Selection:** among candidates passing both, adopt the best by hidden score; else adopt  
 90 nothing (abstain).
- 91 • **Stochastic domains:** replace scores with failure rates over  $N$  runs and require a one-sided  
 92 significant reduction (exact binomial test). Single runs are meaningless when the baseline  
 93 itself is flaky (§4.1).
- 94 • **Scope rule:** a verdict licenses adoption only at the scope of the evaluators that produced it.  
 95 Deployment-scope adoption requires a deployment-scope gate: a screen over the deployable  
 96 distribution, followed by interleaved candidate/baseline confirmation runs on flagged cases  
 97 under a pre-registered decision rule (§4.3).

98 Two design lessons we verified empirically. First, the hidden set must be large or refreshed across  
 99 selection rounds: reusing a 20-item set over 8 candidate selections re-inflated scores by +0.045  
 100 (optimism from gate reuse)—the system-level analogue of adaptive overfitting to a test set. Second,  
 101 the proposer is itself unreliable: models rationalize their failures with their priors instead of fitting the  
 102 evidence, and the induced rules are wrong until the proposer is forced to fit input–output patterns;  
 103 even then, *verification-as-selection*, not proposer quality, is what makes the loop converge. Both  
 104 motivate the central design stance: assume the proposal stream is adversarial.

### 105 3 Fake improvement is the norm

106 **Setup.** Text experiments use a 7B instruct model (Qwen2.5-7B-Instruct) as both proposer and  
 107 substrate; the judge/abstention substrate underneath was validated separately (a zero-shot reading  
 108 judge at 0.949 vs. a tuned guardrail’s 0.789 on a sealed benchmark, with calibrated abstention  
 109 transferring to 4/6 unseen domains). Code experiments execute real Python against held-out unit  
 110 tests.

111 **Fine-tuning level.** Among three candidate LoRA fine-tunes of the judge, the one trained on a single  
 112 source memorizes it (train loss 0.000, in-distribution 0.993) and generalizes worst (held-out 0.507).  
 113 Naive selection (best in-distribution) deploys exactly this memorizer; held-out selection deploys the  
 114 broad candidate (0.578).

115 **Regression is silent.** Fine-tuning to fix one weak source costs another source  $-0.16$  while the target  
 116 barely moves (+0.005); a 1-D “did the target improve?” check misses it entirely. A retention-replay  
 117 candidate removes the regression (worst old-capability  $\Delta = 0.000$ ). This reproduces, on a judge, the  
 118 classic naive-correction backfire—and previews the embodied result of §4.3 in miniature.

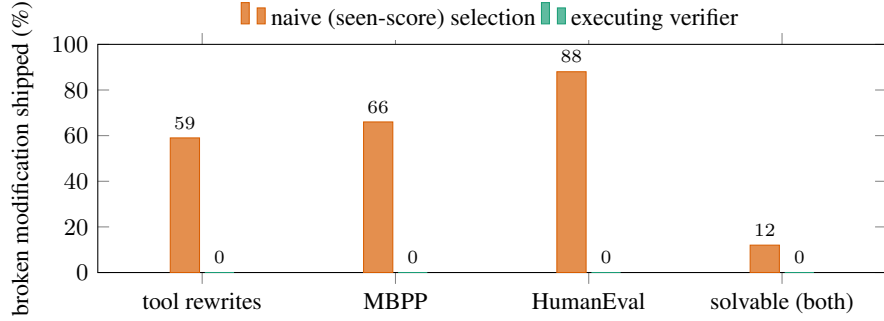


Figure 2: The single adoption decision. When a fake modification exists, naive selection by the score on the data the modification was derived from ships it 59–88% of the time; the verifier (execution on held-out data + regression check) ships 0% across all four settings, while still adopting real improvements (88% of tool decisions; 68% of MBPP problems).

119 **Tool rewrites.** Asked to extend its own Python tool, the agent’s proposal set contains a hardcoded  
 120 lookup of the observed examples in **100% of 24 decisions**; the seen score cannot distinguish it from  
 121 the general tool (both 1.00). Naive selection therefore ships a broken tool **59%** of the time. *Executing*  
 122 candidates on held-out inputs ships **0%**, while still adopting a working tool in 88% of decisions.

123 **Real code.** On MBPP ( $n=100$ ) and HumanEval ( $n=80$ ), a genuine overfit—a solution passing the  
 124 shown test and failing held-out tests—appears on **29% / 30%** of problems respectively. Where it  
 125 exists, naive ships it **19/29** and **21/24** times; the verifier **0 on both**. Restricted to problems the agent  
 126 can solve (removing the abstain-on-unsolvable confound), naive ships a held-out-failing solution  
 127 **12% on both benchmarks**, the executing verifier **0%**. Two independent benchmarks, near-identical  
 128 rates: the effect is a property of the self-modification stream, not of a dataset (Figure 2).

### 129 3.1 The flagship decision, and the autonomous loop

130 An agent with capability A (1.00) and target B (0.00) proposes rule-edits from its own failures. The  
 131 verifier adopts the edit that fixes B *and* keeps A—**rejecting the edit that fixes B best (hidden**  
 132 **1.00) because it regresses A**—ending at A = 1.00, B = 0.83 (mean 0.92) vs. naive 0.67. The  
 133 regression check is not decoration: the best target-fix can be the worst adoption. The loop also closes  
 134 autonomously: forced to fit patterns rather than rationalize, the proposer induces the true hidden rule  
 135 among its candidates and the verifier selects it (2/2 tasks).

### 136 3.2 Honest negative: cumulative compounding

137 Over 3 acquisition rounds the gated agent leads at every round (0.88 vs. 0.79 final). At 5 rounds the  
 138 cumulative-mean advantage shrinks to +0.03: worst-case capability becomes dominated by new-task  
 139 acquisition noise, not by regression. The robust claim in this paper is the *per-decision* advantage;  
 140 large multi-round compounding is not demonstrated on these substrates and we do not claim it.

## 141 4 Embodied: the full arc on a real driving system

142 We instantiate the same gate on CARLA/Bench2Drive [Dosovitskiy et al., 2017, Jia et al., 2024]  
 143 with a real end-to-end student (the LEAD tfv6 transfuser-family stack) and its privileged expert as  
 144 teacher. This section is one continuous, pre-registered narrative: flakiness forces a statistical gate  
 145 (§4.1); the gate rejects a harmful fix and accepts a real one (§4.2); the accepted candidate is then  
 146 *globally* rejected at deployment scope (§4.3); and a ten-recipe repair search under the fixed gate ends  
 147 in a mapped frontier and an honest negative (§4.4).

#### 148 4.1 Failures are flaky; verification must be rate-based

149 Re-running routes with the agent, weather, and configuration unchanged: route 11755 (an  
150 EnterActorFlow scenario) fails  $7/12 = 58\%$  of runs (95% CI  $[0.32, 0.81]$ )—the same route flips  
151 PASS/FAIL run to run—while three routes labeled FAILURE by the single-run benchmark pass 100%  
152 on re-run: their recorded failures were one-off noise. Two consequences. (i) Single-run failure labels,  
153 the leaderboard standard, measure stochasticity as much as capability (run-to-run variance on these  
154 benchmarks is documented [Jia et al., 2024]; we operationalize it into the gate rather than re-discover  
155 it). (ii) Verifying a fix means comparing failure *rates*: confirming a  $58\% \rightarrow 20\%$  improvement needs  
156  $\sim 19$  runs per arm at 80% power. A naive single-run check of an unchanged model would “confirm a  
157 fix” with 42% probability.

158 The first live fix candidate makes the point concrete: a plausible control-layer intervention (“drive  
159 more conservatively”) *raised* the observed failure rate ( $50\% \rightarrow 83\%$ ; collisions persisted and slow  
160 driving added penalties), and the statistical gate rejected it before deployment.

#### 161 4.2 The accepted fix: the loop closes

162 The full cycle then ran end-to-end: student fails 11755 at  $58\% \rightarrow$  the privileged expert is stable-perfect  
163 there ( $8/8$ )  $\rightarrow$  359 MB of expert demonstrations collected on the failure scenario plus retention routes  
164  $\rightarrow$  10-epoch fine-tune from the released checkpoint (single RTX 5090,  $\sim 2.5$  h)  $\rightarrow$  the pre-registered  
165 gate on fresh rollouts: **hidden test** 0/19 failures (baseline  $7/12$ ; exact binomial  $p = 6.0 \times 10^{-8}$ );  
166 **regression** 0/12 on the retention routes  $\rightarrow$  **ACCEPT**. To our knowledge this is the first pre-registered,  
167 error-controlled statistical *accept* produced inside a closed self-improvement loop on an embodied  
168 system—and the same harness had already produced the corresponding *reject* (§4.1).

169 The honest scope of that verdict, stated at the time: verified *same-route repair* over fresh stochastic  
170 rollouts plus a 3-route regression set—not cross-route generalization, and not a full regression sweep.  
171 That scope statement is not a caveat; it is the next experiment.

#### 172 4.3 The deployment gate: same candidate, opposite verdict

173 Is the locally-accepted candidate safe to *deploy*—to swap in globally? The deployment-scope  
174 gate answers with a screen over the 169 baseline-clean routes, then interleaved candidate/baseline  
175 confirmation runs ( $4+4$  per flagged route) under a pre-registered rule (confirmed regression: candidate  
176  $\geq 3/4$  failures and baseline  $\leq 1/4$ ; global reject if any confirmed regression or pooled candidate  
177 failure significantly above baseline).

178 The screen flagged 19 collision routes plus a systematic soft signature (14 routes at exactly score 70  
179 with a red-light violation and mass minimum-speed infractions—a behavioral warp, not noise). The  
180 confirmation batch (176/176 valid interleaved runs) returned: **12 confirmed regressions**; pooled on  
181 flagged routes, candidate  $53/76 = 70\%$  vs. baseline  $5/76 = 7\%$  failures, one-sided  $p = 8.5 \times 10^{-45}$   
182  $\rightarrow$  **GLOBAL REJECT** (Figure 3).

183 The mechanism is classic catastrophic forgetting [McCloskey and Cohen, 1989, French, 1999]: ten  
184 epochs on  $\sim 1.6$ k frames of four scenario types with a 3-route retention set warped global driving—  
185 collisions on ordinary routes, red-light violations, slow driving everywhere. What is new here is not  
186 the phenomenon but the demonstration: *the same candidate earns a legitimate statistical ACCEPT*  
187 *at target scope and a  $p = 8.5 \times 10^{-45}$  REJECT at deployment scope, on a real system, under pre-*  
188 *registered rules*. Single-target evaluation—the standard practice when a fix is verified at all—would  
189 have shipped this model. The 3-route retention check passed (0/12) because those routes happen  
190 to sit among the least-affected; a narrow regression set does not just under-measure damage, it can  
191 systematically miss it. **Verification scope must equal deployment scope**. Nothing was adopted; the  
192 deployed capability floor never dropped.

#### 193 4.4 The ten-recipe repair search: a mapped frontier, an honest negative, a gate that held

194 Can a better fine-tuning recipe pass *both* gates? We ran a pre-registered, timeboxed search: the gate  
195 stays fixed; only the recipe iterates. Local panel (pre-registered lines): fix arm 11755  $\leq 2/8$  failures;  
196 retention arm over the 12 previously-confirmed regression routes, pooled  $\leq 30\%$ ; deployment gate  
197 as in §4.3 for any panel-passer. Ten recipes were evaluated in total (Table 1;  $\sim 450$  verification

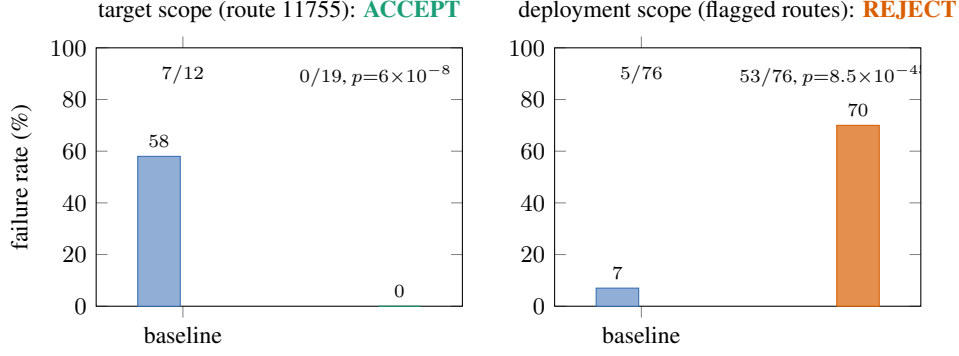


Figure 3: One candidate, two scopes, opposite verdicts—both correct at their scope. Left: the fine-tune genuinely repairs its target route (58%  $\rightarrow$  0% over 19 fresh rollouts). Right: the same model at deployment scope collides on ordinary routes the baseline drives cleanly (12 confirmed regressions; pooled 70% vs. 7%). Single-target evaluation would have shipped it.

Table 1: The repair-search ledger. Every verdict pre-registered; the gate never adopted a regression. Fix line:  $\leq 2/8$  failures on route 11755. Retention line:  $\leq 30\%$  pooled failures on the 12 confirmed-regression routes. <sup>†</sup>K1’s retention was measured at deployment scope (§4.3); panel lines were pre-registered after it.

| #   | recipe (one variable at a time where possible)     | fix 11755                    | retention                                 | verdict                                    |
|-----|----------------------------------------------------|------------------------------|-------------------------------------------|--------------------------------------------|
| 0   | control-layer “drive conservatively” (no training) | worse (50 $\rightarrow$ 83%) | —                                         | REJECT                                     |
| K1  | narrow data, 10 epochs, full FT                    | 0/19 ✓                       | 12 confirmed reg. <sup>†</sup>            | <b>global REJECT</b>                       |
| K2a | K1 data, 3 epochs                                  | 0/8 ✓                        | 12/12 routes fail                         | REJECT                                     |
| K2b | broad retention (17 routes), 3 ep                  | 4/8                          | 33%                                       | REJECT (fix diluted)                       |
| A1  | fix 40% oversample, 3 ep                           | 4/8                          | 25% ✓                                     | REJECT (fix diluted)                       |
| A2  | fix 60% oversample, 5 ep                           | 0/8 ✓                        | 29% ✓                                     | <b>panel PASS <math>\rightarrow</math></b> |
|     | $\hookrightarrow$ deployment gate                  | —                            | 6 confirmed reg., $p=2.5 \times 10^{-20}$ | <b>global REJECT</b>                       |
| A3  | A2 + fix-family demos in fix bucket                | 6/7                          | (cut short)                               | REJECT (family conflict)                   |
| A4  | A2 fix bucket + targeted retention                 | 4/8                          | 33%                                       | REJECT                                     |
| A5  | A4 + frozen backbone (head-only)                   | 4/8                          | $\sim 31\%$                               | REJECT                                     |
| A6  | A2 data + frozen backbone                          | 3/8                          | <b>17% ✓</b>                              | REJECT (fix, by one run)                   |

rollouts): the control-layer intervention, the original narrow fine-tune, and eight variants spanning training length, retention breadth, mixture ratios (via the stack’s native two-bucket oversampling), fix-family breadth, targeted retention collection, and backbone freezing (head-only, 16.6M trainable parameters).

The search converged, honestly, to a mapped trade-off rather than a winner (Figure 4). Its findings:

- **Forgetting here is a data-breadth problem, not a training-length problem:** cutting epochs 10  $\rightarrow$  3 on narrow data preserved the repair and all twelve regressions.
- **The stability–plasticity see-saw is real and sits in the planning head:** with fix data identical to the panel-passing recipe, merely *adding* retention data broke the repair (A4); freezing the perception backbone did not recover it (A5); the interference lives in the shared planning parameters, echoing at system scale the classic trade-off [French, 1999].
- **Dilution is fractal:** adding same-family demonstrations to the fix bucket did not generalize the repair—it diluted and conflicted with it one level down (A3: the family-scenario route itself had failed 4/4 under A2, a within-family overfit; fixing that broke 11755).
- **Panel pass  $\neq$  deployment pass, again:** A2, the one panel-passer, was rejected by the deployment gate (6 confirmed regressions,  $p = 2.5 \times 10^{-20}$ )—though the search was converging (12 confirmed regressions  $\rightarrow$  6, global behavioral warp eliminated).
- **Bounded negative:** at this budget (a single consumer GPU, the stack’s native fine-tuning machinery,  $\sim 450$  verification rollouts), no recipe passed both gates. Full fine-tuning repairs perfectly and collapses globally; head-only training retains best (17%) and misses the fix

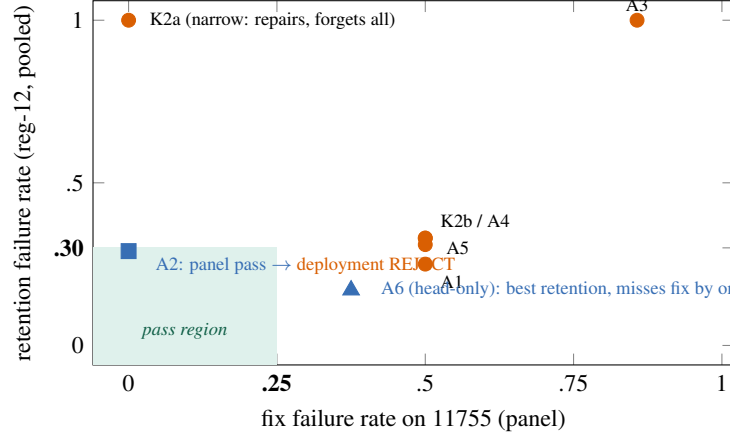


Figure 4: The measured repair–retention frontier across the recipe search (panel scope; A3’s retention arm was cut short and is plotted at its decided axis only). Narrow full fine-tuning repairs and forgets (upper left); breadth or head-only training retains and dilutes (lower right). A2 enters the pass region at panel scope and is then rejected by the deployment gate—the scope rule again. No recipe passes both gates at this budget.

line by one run (3/8). We report this as a first-class result, not a failure of the method under test: the method under test is the *gate*, and the gate did exactly its job.

Through the entire search—ten recipes, every verdict pre-registered—**zero regressions were adopted and the deployed capability floor never dropped**. The gate demonstrated all four of its decision modes on a real system: accept-real (§4.2), reject-harmful (§4.1, §4.3), reject-insufficient (A6), and protect-through-search (this section). A pre-registered caution for practitioners: iterating recipes against a fixed screen risks overfitting the gate itself (the embodied form of the gate-reuse effect of §2); any finally-accepted candidate must additionally pass held-back validation never used during iteration.

## 5 What is and is not established

**Established.** (1) The agent’s own modification stream is adversarial by default—overfits and lookup-hacks are pervasive across surfaces and domains, including real code. (2) Seen/derivation-set score carries no signal against them; held-out (on code, *executed*) evaluation does. (3) The two-gate verifier converts this into a reliable per-decision win: 59% → 0%, 19/29 → 0, 21/24 → 0. (4) Regression checking is necessary: the best target-fix can be the worst adoption. (5) In stochastic embodied domains, only rate-based statistical verification is meaningful, and it supports both accept ( $p = 6 \times 10^{-8}$ ) and reject ( $p = 8.5 \times 10^{-45}$ ) with error control. (6) A verdict is only valid at its measured scope; scope-mismatched acceptance ships catastrophic forgetting on a real system.

**Not established.** Large cumulative multi-round gains (§3.2); verified self-improvement on hard tasks where no candidate transfers (the verifier then correctly adopts nothing—safe, but not improvement); a deployment-gate-passing embodied repair (§4.4: bounded negative at this budget); behavior beyond a 7B proposer, benchmark-scale substrates, one vehicle stack, and one simulator.

## 6 Related work and claim boundaries

**Self-improving agents** (Reflexion [Shinn et al., 2023]; ExpeL [Zhao et al., 2024]; AutoGuide [Fu et al., 2024]; Voyager [Wang et al., 2024]) supply the proposer we gate. **That self-generated improvement is frequently fake is established:** agent benchmarks are reward-hackable [Berkeley RDI, 2026]; frontier agents hack their evaluations spontaneously [METR, 2025]; a self-modifying system falsified its own metric [Zhang et al., 2025]; self-reflection confabulates. **Gating self-modification is an emerging thread:** PACE applies anytime-valid acceptance tests to prompt/strategy updates on text tasks [PACE authors, 2026]; GRACE proposes an update-rejection gate [GRACE authors, 2026];

sequential statistics have been brought to robot policy comparison offline [Beyond-Binary-Success authors, 2025]. **Driving-benchmark noise is documented** [Jia et al., 2024, Dosovitskiy et al., 2017], with published scores typically single-run. **Continual learning** names our regression phenomenon catastrophic forgetting [McCloskey and Cohen, 1989, French, 1999, Kirkpatrick et al., 2017], and our repair recipes (replay ratios, freezing, DAgger-style collection [Ross et al., 2011, Hu et al., 2022]) are its textbook tools deliberately reused, not claimed as new.

We therefore *do not* claim: the first observation that self-improvement is often fake; the first statistical gate for self-improving agents; or the discovery of driving-benchmark flakiness. To our knowledge, what *is* unclaimed before this work: (a) fake-improvement *rates measured across all four modification surfaces* under one protocol, plus two real code benchmarks; (b) a *pre-registered, error-controlled statistical accept and reject operating inside a closed self-improvement loop on an embodied system*, including the operationalization of run-to-run noise into a failure-rate gate; and (c) the *deployment-scope demonstration*—a narrowly-scoped gate accepts a change whose collateral damage a scope-matched gate then catches, on a real system, followed by a gate-protected recipe search ending in a mapped frontier. (Literature scan of July 2026; the space moves fast.)

## 7 Limitations

A 7B proposer; controlled/benchmark substrates; six-item held-out sets on the toy loops (granularity 0.17); one vehicle stack and one simulator; the embodied fix targets one (representative, flakiest) failure route with an 8-run fix arm and 24-run retention arm at panel scope—adequate for the pre-registered lines, underpowered for small effects; the bounded negative of §4.4 is a statement about this training budget and recipe family (parameter-efficient adapters and larger fix-side data remain untested). Gate reuse must be budgeted (§2); we spec but do not automate it. Multi-round compounding may appear on richer substrates; we did not demonstrate it.

## 8 Conclusion

Across text, real code, and embodied driving, self-improvement without verification ships fakes at rates of 12–88%, and the fakes are supplied by the loop itself. A verifier that demands held-out transfer and no regression—nothing more exotic—reduced shipped fakes to zero in our experiments while still adopting real improvements; its statistical form both accepted a real embodied repair and rejected the same candidate at deployment scope before it could ship catastrophic forgetting; and under a ten-recipe search it held the capability floor while honestly reporting that no recipe earned adoption. If self-improving agents are to be trusted with their own modification stream, the verifier is not a component of the loop. **The verifier is the loop; the rest is a proposal generator.**

## References

- Berkeley RDI. Agent benchmarks are reward-hackable: Exploits against eight major evaluations. Technical report, UC Berkeley Center for Responsible Decentralized Intelligence, 2026. VERIFY-BEFORE-SUBMISSION: exact title, author list, and report/arXiv ID.
- Beyond-Binary-Success authors. Beyond binary success: Sequential statistical comparison of robot policies. *arXiv preprint*, 2025. VERIFY-BEFORE-SUBMISSION: author list and arXiv ID.
- Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, 2017.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.



GRACE authors. GRACE: Gated rejection of agent capability edits. *arXiv preprint*, 2026. VERIFY-BEFORE-SUBMISSION: author list, exact title, and arXiv ID.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Xiaosong Jia, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

METR. Spontaneous reward hacking by frontier agents in evaluation environments. Technical report, METR, 2025. VERIFY-BEFORE-SUBMISSION: exact title and report URL.

PACE authors. PACE: Anytime-valid acceptance tests for prompt and strategy updates in deployed language systems. *arXiv preprint*, 2026. VERIFY-BEFORE-SUBMISSION: author list and arXiv ID (scan of 2026-06).

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

## A Reproducibility

All numbers in this paper are produced by scripts in the accompanying repository (anonymized for review); each result entry in the repository’s ledger names the script and exact configuration that produced it. Text and code experiments run on a single GPU with a 7B open-weight model; embodied experiments use CARLA 0.9.15 + Bench2Drive with the LEAD stack on a single RTX 5090 (Windows-native). The gate itself is released as a dependency-free library (`vsi_gate.py`) exposing `gate()` (score form) and `gate_rate()` (exact-binomial failure-rate form), with a deterministic no-LLM demo.

## B Pre-registered decision rules (embodied)

Fixed before the corresponding runs, unchanged throughout: **(local panel)** fix arm: route 11755, 8 fresh rollouts, pass iff  $\leq 2$  failures; retention arm: the 12 confirmed-regression routes  $\times 2$ , pass iff pooled failures  $\leq 30\%$ ; both required. **(deployment gate)** screen all 169 baseline-clean routes

340 once; flag any failure; triage pure-artifact flags (minimum-speed/lane-only) out; confirm each flagged  
341 route with interleaved candidate/baseline runs (4+4); confirmed regression iff candidate  $\geq 3/4$   
342 failures and baseline  $\leq 1/4$ ; global REJECT iff  $\geq 1$  confirmed regression or pooled candidate failure  
343 significantly above baseline (one-sided exact test,  $\alpha = 0.05$ ). (**rate gate**) a fix claim requires a  
344 one-sided exact-binomial significant reduction against the measured baseline rate, power-planned  
345 before collection (e.g. 58%  $\rightarrow$  20% requires  $\sim 19$  runs/arm at 80% power).